

 Member-only story

NEW Docker 2025 : 42 Prod Best Practices The Complete Guide for Developers



Mahernaija

[Follow](#)

8 min read · Jan 15, 2025



47



Must-know best practices for faster, more secure, and scalable applications!

Introduction

Master the art of modern software development with Docker and Docker Compose! 🚀 These essential tools empower developers to build, manage, and scale applications effortlessly in isolated environments. In this guide, explore the latest best practices for 2024 and learn how to deploy a FastAPI service using Docker like a pro.

Best Practices List:

Check below for details : how to implement, benefests ...

Docker Best Practices:

1. Use Minimal Base Images 🖌️
2. Use Official Base Images 🏗️
3. Leverage Multi-Stage Builds 🏗️
4. Minimize Layers 🧱
5. Use .dockerignore 📁
6. Use Specific Version Tags 🏷️
7. Clean Up After Installation 🖌️
8. Limit Container Privileges 🔒
9. Specify a Health Check 🛡️

Open in app ↗



Medium



Search



Write



12. Minimize Image Size 📦

13. Avoid Hard-Coding Ports 🏠

14. Use Signals Correctly in ENTRYPOINT ↺

15. Log Verbosely for Easier Debugging 🔧

16. Set Permissions Correctly 🛡️

17. Always Use Immutable Image Tags 🏷️

18. Optimize Docker Cache Layers 🧱

19. Use Signed Images 🔑

20. Encrypt Secrets and Sensitive Data 🔒

21. Use Environment Variables 🌐
22. Document Dockerfile and Container Usage 📖
23. Log Outputs Clearly 🖥️
24. Handle Signals Correctly for Graceful Shutdown ↺

Docker Compose Best Practices:

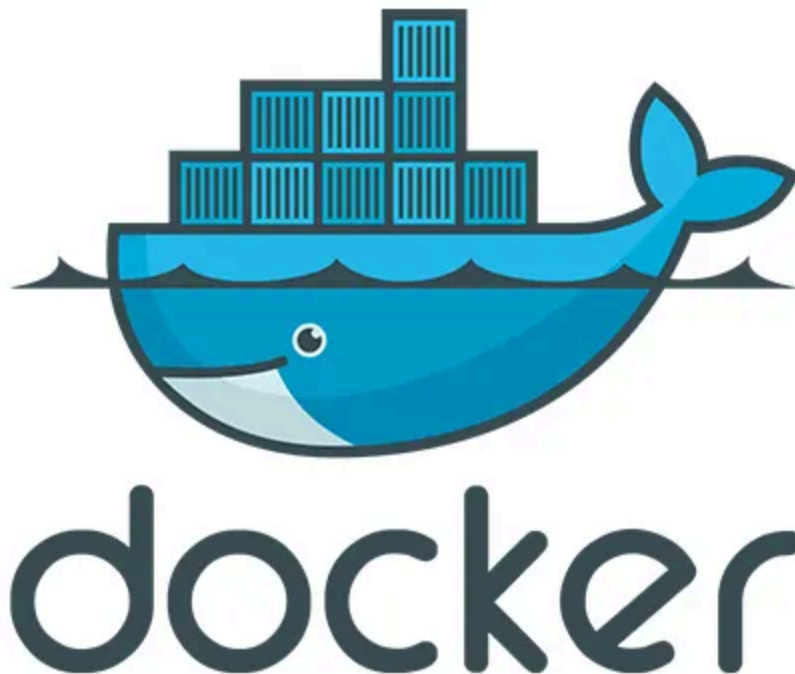
1. Use .env Files for Configuration 🔑
2. Monitor Container Health 🛡️
3. Limit Resources for Better Performance ⚖️
4. Installing Environment-Specific Dependencies ⚙️
5. Use Docker Compose's Watch Feature for Development 🖥️
6. Use Volumes for Persistence 💾
7. Enable Networking Isolation 📡
8. Define Restart Policies ♻️
9. Deploy with Replicas (Scaling) 📈
10. Limit Container Privileges 🔒
11. Enforce Image Pull Policies 🎯
12. Run Containers in Non-Privileged Mode 🚫
13. Use Named Volumes and Networks 📁
14. Use Service Dependencies 🔧
15. Log Configuration 🖥️
16. Use External Configuration Files 📄

17. Use Labels for Metadata 🏷️

18. Test Before Deploying 🧪

What is Docker? 🐳

Docker is a containerization platform that packages applications and their dependencies, ensuring consistent behavior across all environments. 🛠️ It eliminates “it works on my machine” errors, whether on local or cloud deployments. 🛠️



What is Docker Compose? 📜

Docker Compose simplifies managing multi-container applications by defining services in a single YAML file and controlling them with one command. 📁 It's essential for handling complex architectures where containers need to communicate seamlessly.

Prerequisites

Docker

- Install **Docker version 27.3.1, build ce12230** and ensure the Docker daemon is running. You can follow the official guide for installation: [Docker Installation](<https://docs.docker.com/engine/install/>)

Docker Compose

- Install **Docker Compose version v2.29.6** by following the official installation guide: [Docker Compose Installation](<https://docs.docker.com/compose/install/standalone/>)

Best Practices for Docker & Docker Compose in 2025 ✓

To help you optimize your Docker workflows and container management, let's dive into some essential best practices.

1. Use Minimal Base Images 🖌️

How to Implement:

```
FROM python:3.9-slim
```

2. Use Official Base Images

How to Implement:

- Use well-maintained official base images like `alpine`, `debian-slim`, or `ubuntu`. For example:

```
FROM python:3.9-alpine
```

3. Leverage Multi-Stage Builds 🏗️

How to Implement:

```
# Stage 1: Build stage
FROM python:3.9.20-slim AS builder
ARG ENVIRONMENT=dev
WORKDIR /app/
COPY requirements/ /app/requirements/
RUN if [ "$ENVIRONMENT" = "prod" ]; then \
    pip install --no-cache-dir -r /app/requirements/prod.txt --target /app/deps \
else \
    pip install --no-cache-dir -r /app/requirements/dev.txt --target /app/deps \
fi
```

```
# Stage 2: Final runtime stage
FROM python:3.9.20-slim
COPY --from=builder /app/deps /usr/local/lib/python3.9/site-packages
COPY ./src /app
```

3. Minimize Layers

How to Implement:

- Each Dockerfile instruction creates a new layer. Combine multiple instructions into a single `RUN` when possible, and chain commands using `&&`.

```
RUN apt-get update && apt-get install -y \  
    curl \  
    vim \  
    && rm -rf /var/lib/apt/lists/*
```

4. Use .dockerignore File

How to Implement:

- Exclude unnecessary files and directories from the Docker context using a `.dockerignore` file, reducing build size and time.

5. Set WORKDIR

How to Implement:

- Use `WORKDIR` to set the working directory instead of using `RUN cd ...`. It makes the Dockerfile cleaner and reduces layer creation.

```
WORKDIR /app
```

7. Use Specific Version Tags

How to Implement:

- Pin specific versions of base images and packages to ensure consistent builds.

```
FROM node:16.13.1-alpine
```

8. Clean Up After Installation

How to Implement:

- Remove cache, temp files, and unnecessary dependencies after package installations to keep the image small.

```
RUN apt-get update && apt-get install -y \    build-essential \    && rm -rf /
```

9. Limit Container Privileges

How to Implement:

- Use a non-root user for running the application to minimize security risks. Create the user using `RUN` and switch with `USER`.


```
RUN addgroup -S appgroup && adduser -S appuser -G appgroup USER appuser
```

10. Specify a Health Check

How to Implement:

- Add a `HEALTHCHECK` instruction to verify that your application is running correctly. This helps Docker manage your containers effectively.

```
HEALTHCHECK CMD curl --fail http://localhost:8080/health || exit 1
```

11. Use `CMD` and `ENTRYPOINT` Appropriately

How to Implement:

- Use `CMD` to define default arguments and `ENTRYPOINT` to define the default executable. This provides flexibility while keeping the container usable for different scenarios.

```
ENTRYPOINT ["python", "app.py"] CMD ["--help"]
```

12. Label Your Image

How to Implement:

- Use `LABEL` to provide metadata, such as the maintainer's email or version information.

```
LABEL maintainer="maher.naija@gmail.com"
```

13. Minimize Image Size

How to Implement:

- Use `--no-cache` and remove development dependencies where possible.

```
RUN pip install --no-cache-dir -r requirements.txt
```

14. Avoid Hard-Coding Ports

How to Implement:

- Expose ports dynamically using environment variables rather than hard-coding them.

```
EXPOSE ${PORT:-8080}
```

15. Use Signals Correctly in ENTRYPOINT

How to Implement:

- Set `ENTRYPOINT` with the `exec` form to ensure the application can handle signals like `SIGTERM` properly for graceful shutdown.

```
ENTRYPOINT ["exec", "myapp"]
```

16. Log Verbosely for Easier Debugging

How to Implement:

- Add logging to the build process to simplify troubleshooting.

```
RUN echo "Building app..." && \    echo "Step 1: Installing dependencies"
```

17. Set Permissions Correctly

How to Implement:

- Set file permissions correctly when copying files to ensure they can be executed or accessed as needed.

```
COPY --chown=appuser:appgroup myapp /usr/local/bin/
```

18. Always Use Immutable Image Tags

How to Implement:

- When deploying, use immutable tags (like `:v1.2.3`) instead of `:latest` to avoid pulling unexpected image versions.

19. Optimize Docker Cache Layers

How to Implement:

```
COPY requirements.txt /app/  
RUN pip install -r /app/requirements.txt  
COPY . /app/
```

This ensures that dependency installation is cached separately, speeding up future builds.

20. Use Signed Images

How to implement

Verify the authenticity and integrity of the base images you use by enabling Docker Content Trust. This ensures the images come from trusted sources.

```
export DOCKER_CONTENT_TRUST=1
```

21. Encrypt Secrets and Sensitive Data

- Never store sensitive information like passwords or tokens in Dockerfiles. Use Docker environment variables to inject them securely at runtime.

22. Use Environment Variables

- Use environment variables to configure your application dynamically. This makes the Dockerfile flexible and easier to maintain for different environments (development, staging, production).

```
ENV NODE_ENV=production
```

Document Dockerfile and Container Usage

- Add comments to your Dockerfile to explain the purpose of each instruction and include documentation for how to run and maintain the container.

23. Log Outputs Clearly

- Ensure your application logs are outputted to `stdout` and `stderr` to allow easy integration with Docker's logging drivers and external monitoring tools.

24. Handle Signals Correctly

- Ensure that your application correctly handles OS signals like `SIGTERM` and `SIGINT`, allowing your containers to be stopped gracefully during maintenance or scaling operations.

25. Use hadolint for linting

- lint your docker file using hadolinter

Docker-Compose practices:

1. Use .env Files for Configuration

How to Implement:

```
services:
  db:
    image: postgres:12.1-alpine
    environment:
      - POSTGRES_USER
      - POSTGRES_PASSWORD
      - POSTGRES_DB
```

2. Monitor Container Health

How to Implement:

```
services:
  api:
    image: fastapiapp:latest
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 10s
      retries: 3
```

3. Limit Resources for Better Performance

How to Implement:

```
services:
  api:
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 512M
```

This configuration ensures services have dedicated resources, improving stability.

4. Installing Environment-Specific Dependencies

How to Implement:

In multi-environment setups (e.g., development, staging, production), it's essential to install environment-specific dependencies.

```
RUN if [ "$ENVIRONMENT" = "prod" ]; then \
    pip install --no-cache-dir -r /app/requirements/prod.txt; \
else \
    pip install --no-cache-dir -r /app/requirements/dev.txt;
```

5. Use Docker Compose's Watch Feature for Development

How to Implement:

```
develop:
  watch:
    - action: sync
      path: ./src/
      target: /app/src/
    - action: rebuild
      path: requirements/dev.txt
```

6.Use Volumes for Persistence

- Use volumes to persist data across container restarts, ensuring availability of data such as databases or configuration files.

```
services:
  db:
    image: postgres
    volumes:
      - db-data:/var/lib/postgresql/data
```

```
volumes:
  db-data:
```

7.Enable Networking Isolation in Docker Compose

How to Implement:

- Use Docker Compose networks to isolate services from each other. Only allow necessary communication between services.


```
networks:
  app_network:
    driver: bridge
services:
  app:
    networks:
      - frontend
  db:
    networks:
      - backend
networks:
  frontend:
  backend:
```

8. Define Restart Policies

How to Implement:

- Use `restart` policies in your `docker-compose.yml` or container orchestration settings to ensure your containers restart automatically on failure.

```
restart: always
```

9. Deploy with Replicas (Scaling)

How to Implement:

- Define replicas to scale services horizontally. This ensures high availability by running multiple instances of services.

```
services:
  app:
    image: myapp
    deploy:
      replicas: 3
```

11.Limit Container Privileges

How to Implement:

- Use `cap_drop` to remove unnecessary Linux capabilities from the container and `read_only` to prevent write access to the filesystem.

```
services:
  app:
    image: myapp
    cap_drop:
      - ALL
    read_only: true
```

12.Enforce Image Pull Policies

How to Implement:

- Use `image:tag` and set `pull_policy` to `always` for important services to make sure the latest image is always pulled from the registry, ensuring up-to-date versions with security patches.

```
services:
  app:
```

```
image: myapp:1.0.0  
pull_policy: always
```

13.Run Containers in Non-Privileged Mode

How to Implement:

- Avoid running containers with elevated privileges by not using `privileged: true`. This ensures that even if the container is compromised, it can't damage the host.

```
services:  
  app:  
    image: myapp  
    privileged: false
```

14.Use Named Volumes and Networks

How to Implement:

- Use named volumes and networks for clarity and reuse across services, making your setup easier to manage and debug.

```
services:  
  app:  
    volumes:  
      - app-data:/var/www/html  
volumes:  
  app-data:
```

15. Use Service Dependencies

How to Implement:

- Define `depends_on` to control the startup order of services and ensure that dependent services like databases are running before your application starts.

```
services:
  app:
    depends_on:
      - db
```

17. Log Configuration

How to Implement:

- Use external logging drivers to forward container logs to a centralized location like Logstash, Fluentd, or AWS CloudWatch for easier management and debugging.

```
services:
  app:
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

18. Use External Configuration Files

How to Implement:

- Use external configuration files for services (e.g., Nginx, Apache) to make container updates easier and more manageable.

```
services:
  web:
    image: nginx
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
```

19. Use Labels for Metadata

How to Implement:

- Add labels for better tracking and manageability of services, especially in production or multi-team environments.

```
services:
  app:
    labels:
      - "maintainer=maher.naija@gmail.com"
      - "version=1.0"
```

20. Testing Before Deploying

- Run `docker-compose config` to validate the configuration and catch syntax errors before deploying.

```
docker-compose config
```

Conclusion: Go Prod Like a Pro! 🚀

🚨 Don't miss out! 🚨

<https://www.benchhub.co/>

☀️ It's full of insights and everything you need to know! 📝 ✨

Mastering Docker and Docker Compose in 2025 is essential for deploying fast, secure, and scalable applications. By following these best practices, you can optimize your workflows, reduce risks, and ensure your app runs smoothly in any environment. 🚀

Ready to elevate your app to new heights? With these strategies in place, your applications will be future-proof, efficient, and scalable! Let's build the future, one container at a time! 🛠️ ✨

Docker +

Docker Compose +

DevOps +

Devops Tool +

Kubernetes +



Written by Mahernaija

616 followers · 494 following

Devops engineer / solution architect

Follow

